

Range Queries (and Much More...)

Design and Analysis of Algorithms

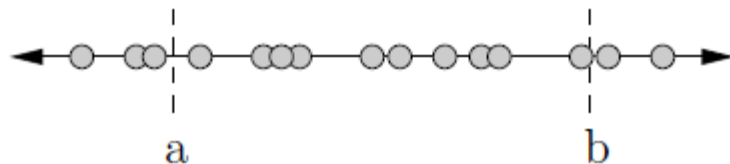
Brian C. Dean

**School of Computing
Clemson University**

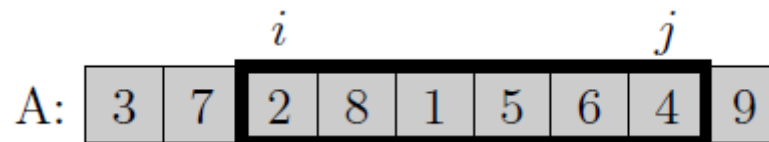


Range Queries : Examples

- One-dimensional queries (over a set or sequence):

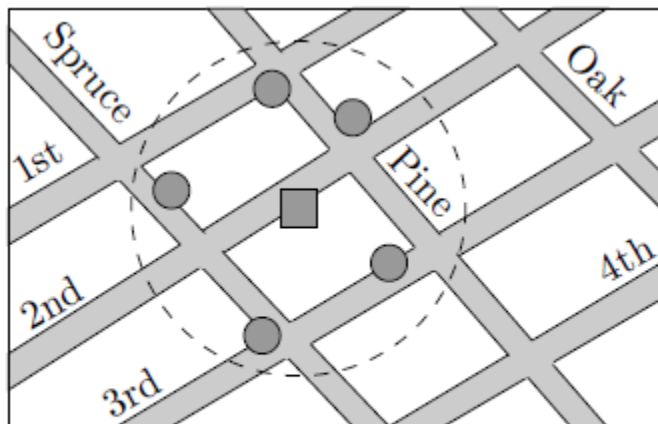


“Enumerate all points in the range $[a, b]$.”
“How many points lie in the range $[a, b]$?”

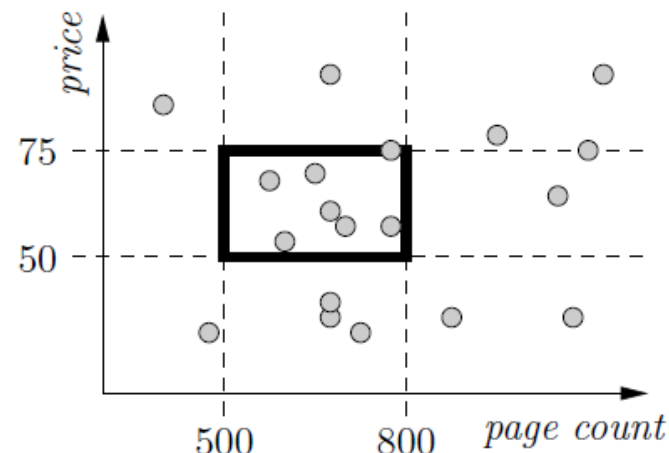


“What is the sum of $A[i \dots j]$?”
“What is the minimum of $A[i \dots j]$?”

- Multi-dimensional queries:



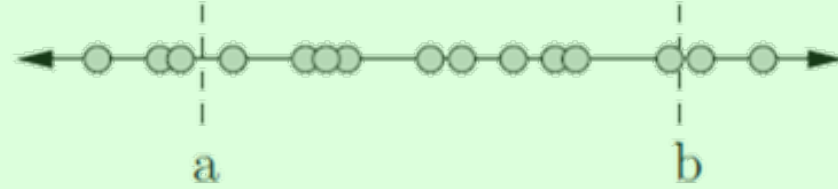
“How many restaurants are within 1 km of my present location?”



“How many books have a price in $[50, 75]$ and a page count in $[500, 800]$?”

Static Versus Dynamic Problems

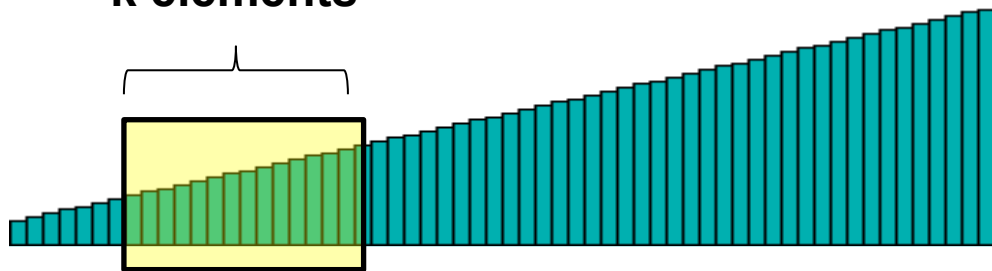
Example: 1D Range Queries in a Set



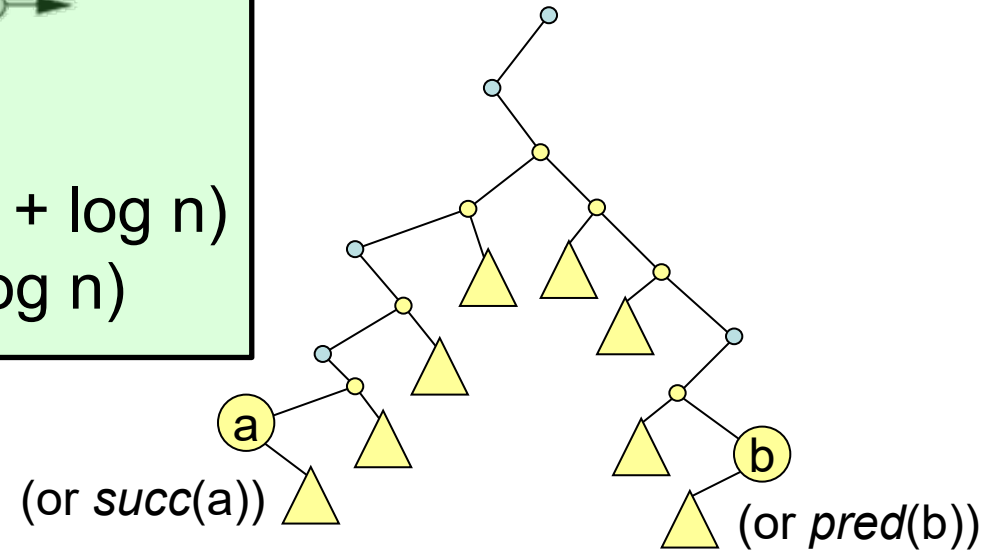
Enumerate k points in $[a, b]$: $O(k + \log n)$

Count points in $[a, b]$: $O(\log n)$

k elements

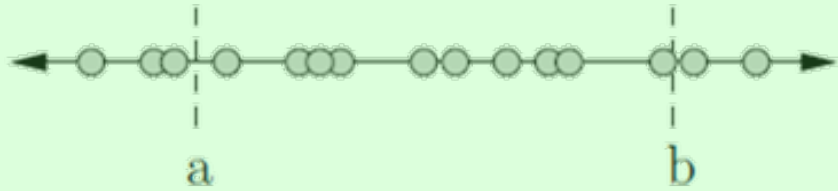


Static: preprocess to build data structure ($O(n \log n)$ here via sorting), then handle queries.



Dynamic: queries interspersed with *inserts* and *deletes* (which also take $O(\log n)$ time here if using a balanced BST)

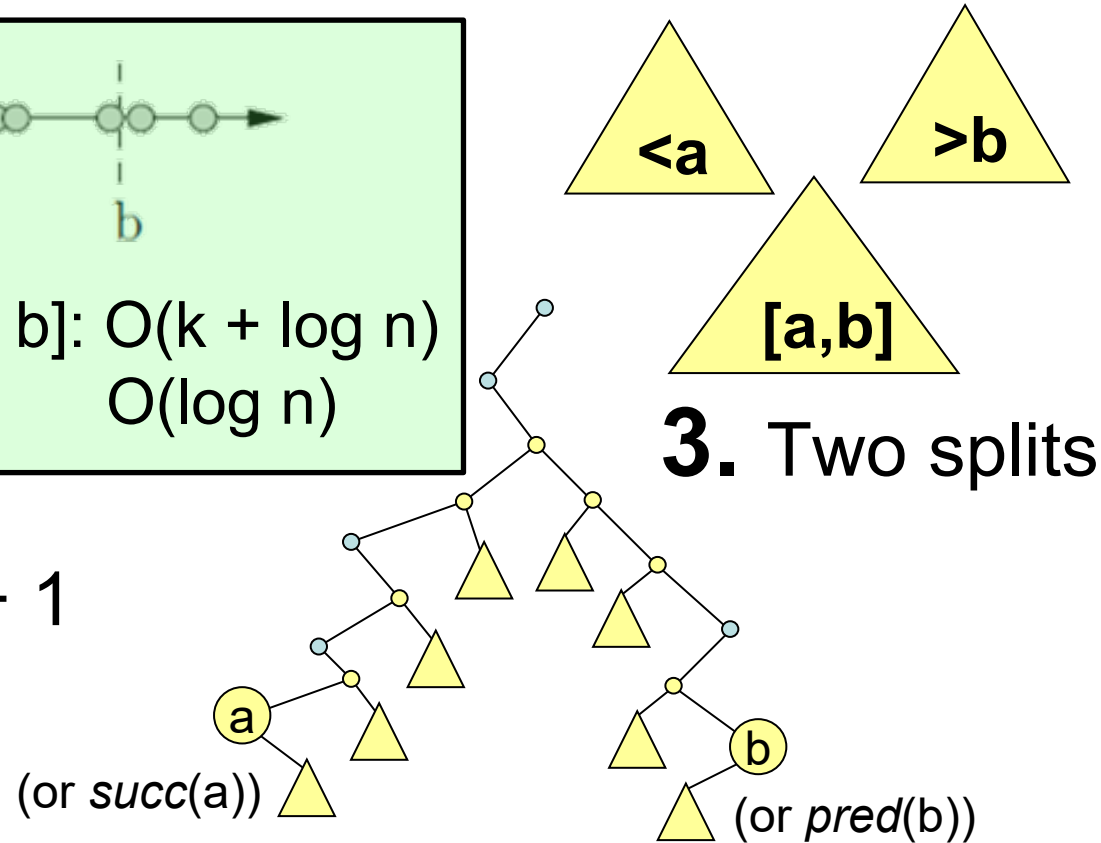
Range Queries in a Balanced BST



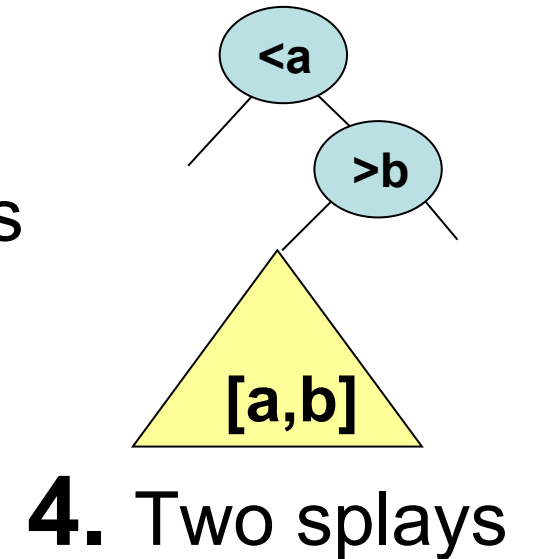
Enumerate k points in $[a, b]$: $O(k + \log n)$
Count points in $[a, b]$: $O(\log n)$

1. $\text{rank}(b) - \text{rank}(a) + 1$
(adjust carefully
if a or b not in tree)

2. Successive successors
(fun fact: $O(k + \log n)$ rather
than $O(k \log n)$ for enumeration)

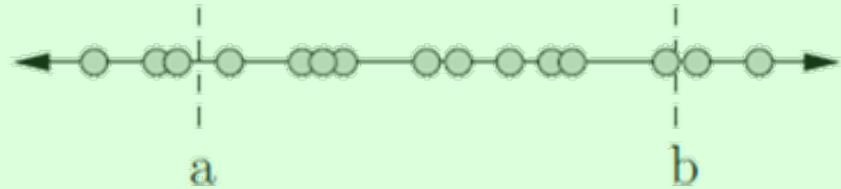


3. Two splits



4. Two splits

Range Queries in BSTs via Pruned Traversals



Enumerate k points in $[a, b]$: $O(k + \log n)$

Count points in $[a, b]$: $O(\log n)$

RangeQuery(RootNode N , Range R) :

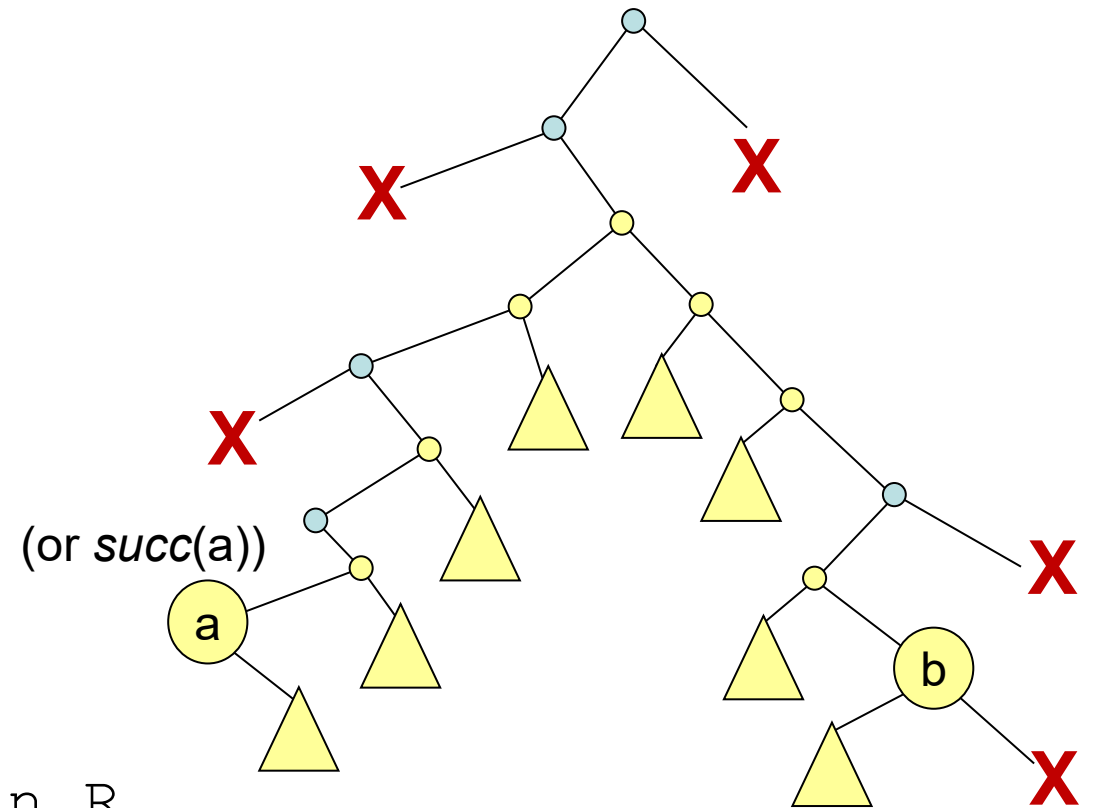
If N is NULL, return

If N disjoint from R , return

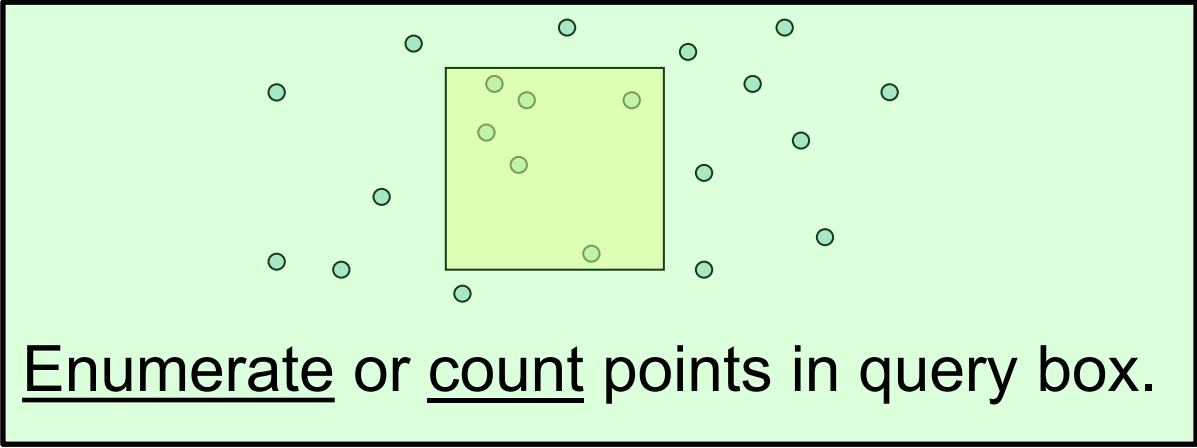
**If counting and N inside R ,
return N 's subtree size**

Include N 's value in answer if in R

Recursively query N 's children



Pruned Traversal Works in ANY Recursive Spatial Decomposition...



Enumerate or count points in query box.

RangeQuery(RootNode N, Range R) :

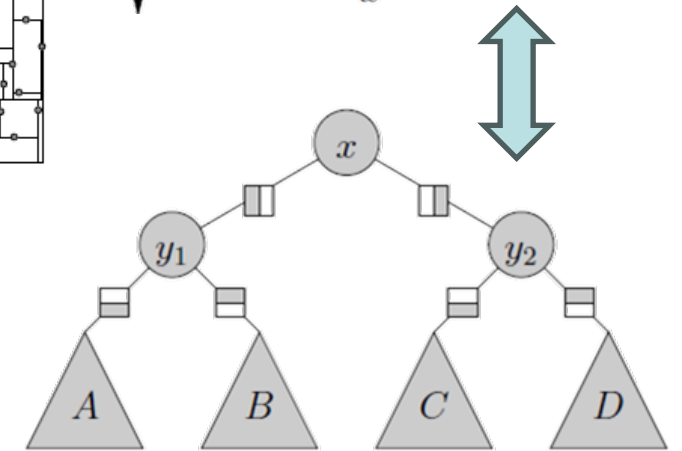
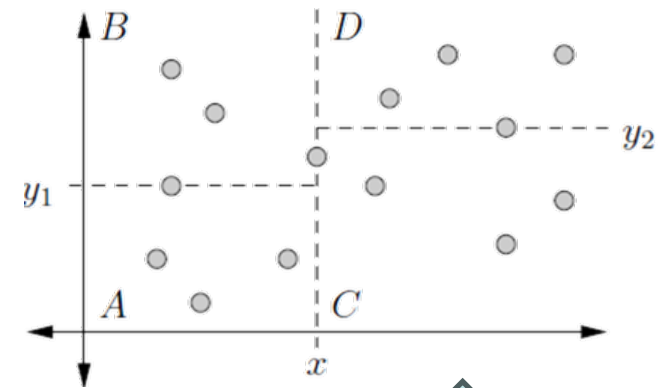
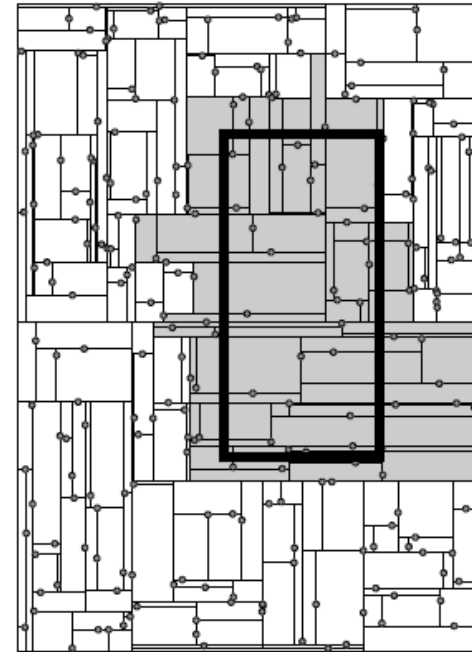
If N is NULL, return

If N disjoint from R, return

**If counting and N inside R,
return N's subtree size**

Include N's value in answer if in R

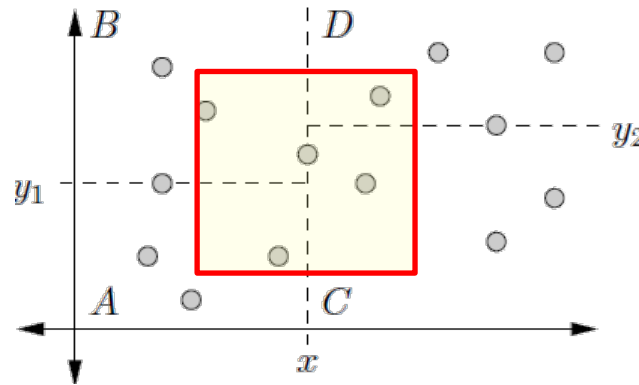
Recursively query N's children



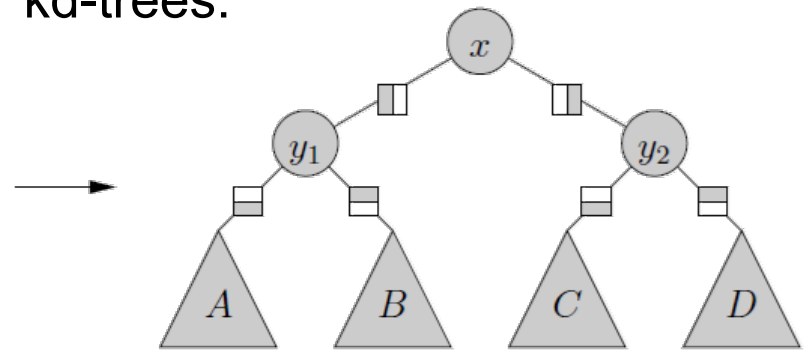
A kd-tree

Orthogonal Range Query Performance in Higher Dimensions

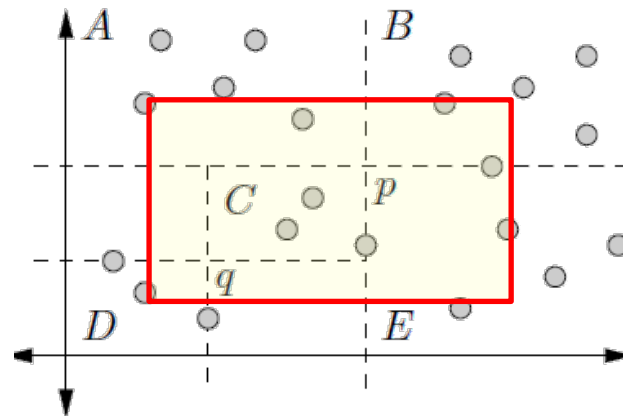
$O(k + \sqrt{n})$ enumeration
 $O(k + n^{1-1/d})$ in d dims
(no $+k$ term if counting)
(assume perfect balance)



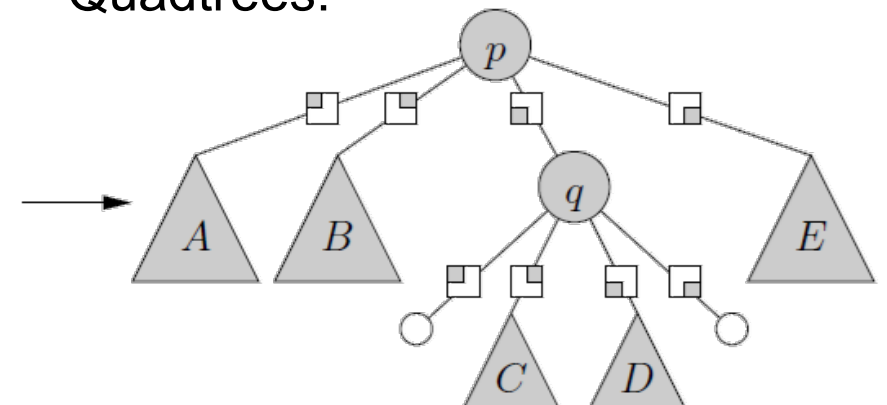
kd-trees:



$O(n)$ ☹️
(maybe still ok in practice though...)



Quadtrees:

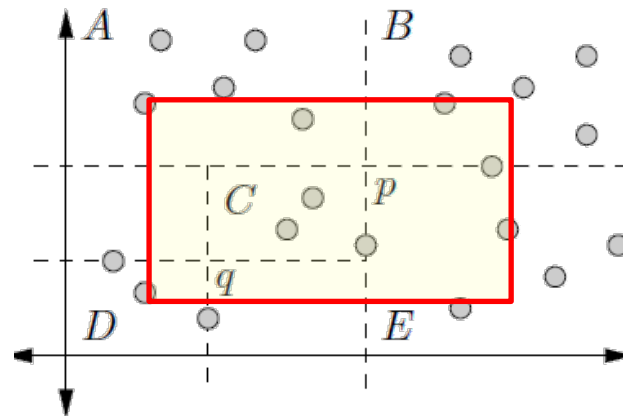
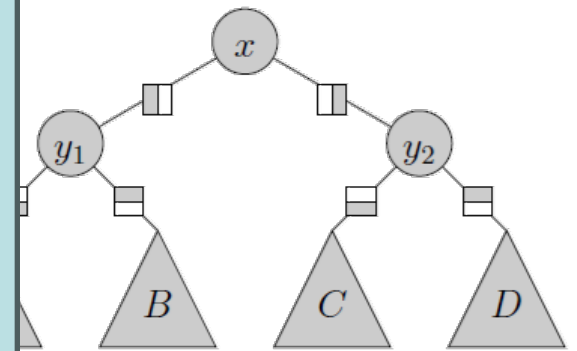


Orthogonal Range Query Performance in Higher Dimensions

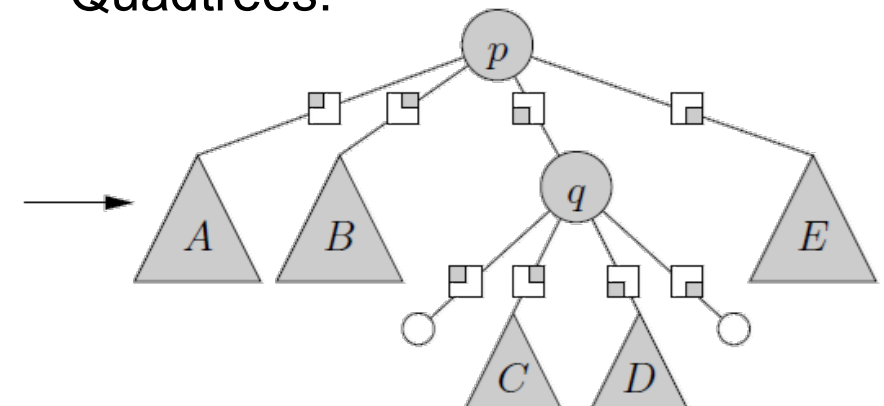
$O(k + \sqrt[n]{n})$ enumeration
 $O(k + n^{1-1/d})$ in d dims
(no $+k$ term if counting)
(assume perfect balance)

$O(n)$ ☹️
(maybe still ok in practice though...)

If randomly balanced,
 $O(k + n^a)$ in 2D, where
 $a = (\sqrt{17} - 3) / 2 \approx 0.562$



Quadtrees:

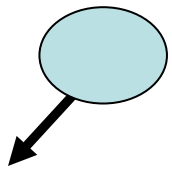


Why $\sqrt{n}$?

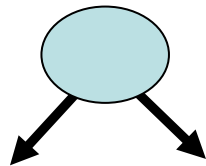
(Assuming Tree Perfectly Balanced...)

Only leaf nodes on boundary “matter”. And only $O(\sqrt{n})$ of them.

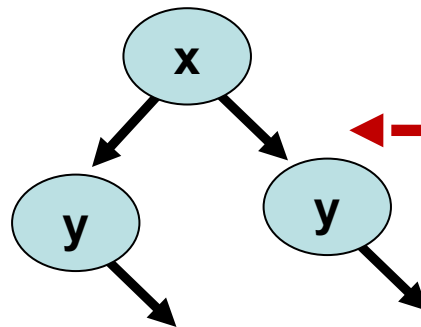
Recurse one way: $O(\log n)$



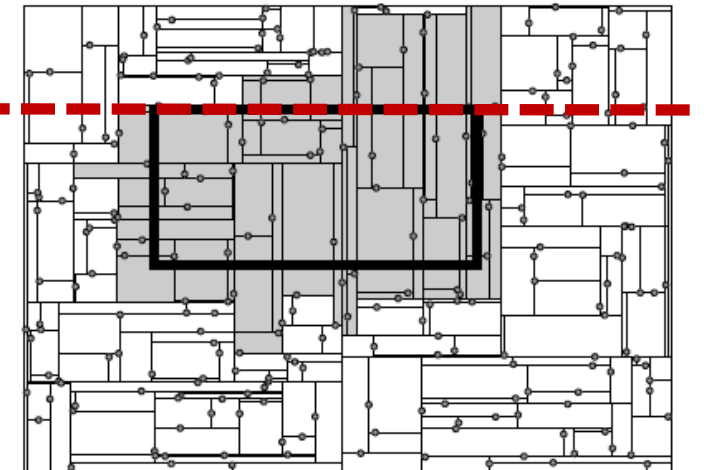
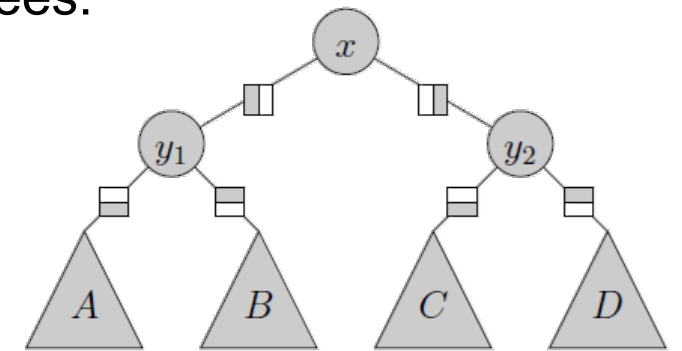
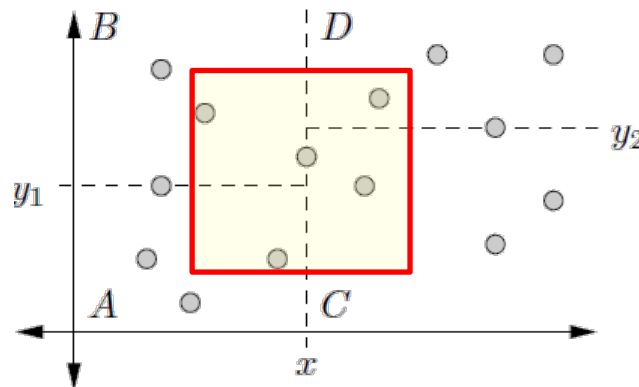
Recurse both ways: $O(n)$
(a.k.a. $2^{\log_2 n}$)



Alternating: $O(\sqrt{n})$
(a.k.a. $2^{\frac{1}{2} \log_2 n}$)



kd-trees:



Recall: Viewing Databases as Multi-Dimensional Sets of Points

- Structured Query Language (SQL) is a common way to interact with databases:

```
SELECT title, author FROM books_in_library
      WHERE price <= 100                                (10 million matches)
      AND publication date >= 1990                       (100 million matches)
      AND page_count BETWEEN 500 AND 750;               (100,000 matches)
```

- This example corresponds to a range query over a 3D rectangle.
- Queries like this can be sped up by telling the database to build one-dimensional indexes (often B-trees) on specific fields.
 - E.g., using an index on `page_count`, we could quickly identify all 100,000 records matching the $500 \leq \text{page_count} \leq 750$ range and then test them one by one (slowly) against the other conditions.